

関数

Kotlinで関数を宣言するには：

- `fun` キーワードを使用してください。
- パラメータは括弧 `()` で指定してください。
- 必要に応じて、[戻り値の型](#) を記載してください。

例えば、与えられた整数型の値いを2倍にして戻す関数`twice()`を考えます。：

```
//sampleStart
// 「twice」は関数の名前です
// 『x』 は Int 型のパラメータです
// 期待される戻り値も Int 型です
fun twice(x: Int): Int {
    return 2 * x
}
//sampleEnd

fun main() {
    println(twice(5))
    // 10
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" validate="false" id="kotlin-function-double"}
```

Function usage

関数の使い方

関数の呼び出しは、標準的な方法で行います：

```
val result = twice(2)
```

[メンバー](#) や [拡張関数](#) を呼び出すには、ピリオド `.` を使用します：

```
// Streamクラスのインスタンスを作成し、read()を呼び出します
Stream().read()
```

Parameters

パラメータ

関数の引数は、パスカル表記 (`name: Type`) で宣言してください。パラメータはコンマで区切り、各パラメータの型を明示的に指定する必要があります：

```
fun powerOf(number: Int, exponent: Int): Int { /*...*/ }
```

関数にオブジェクトを渡す際、コンパイラはそのオブジェクトへの参照のコピーを渡します。コピーされた参照は同じオブジェクトを指しているため、この関数はそのオブジェクトの可変状態を変更することができます。

関数内のパラメータは関数本体内では読み取り専用（暗黙的に `val` として宣言される）であるため、それらに新しい値を代入することはできません：

```
class Counter(var value: Int)

fun reset(counter: Counter) {
    counter.value = 0 // Allowed: modifies the object
    counter = Counter(0) // Error: 'val' cannot be reassigned
}
```

関数のパラメータを宣言する際には、[末尾のコンマ\(トレーリングコンマ\)](#)を使用できます：

```
fun powerOf(
    number: Int,
    exponent: Int, // trailing comma
) { /*...*/ }
```

末尾のコンマは、リファクタリングやコードの保守に役立ちます：宣言内でパラメータの順序を自由に変更でき、どのパラメータが最後になるかを気にする必要はありません。

Kotlinの関数は、他の関数をパラメータとして受け取ることができ、また引数として渡すこともできます。詳細については、[ラムダ式](#)を参照してください。

{style="note"}

Parameters with default values {id="parameters-with-default-values"}

デフォルト値を持つパラメータ {id="parameters-with-default-values"}

関数のパラメータには、デフォルト値を指定することで、そのパラメータをオプションにすることができます。Kotlinでは、そのパラメータに対応する引数を指定せずに関数を呼び出した場合、デフォルト値が使用されます。デフォルト値を持つパラメータは、「オプションパラメータ」とも呼ばれます。

オプションのパラメータがあれば、妥当なデフォルト値を持つパラメータを省略できるようにするためだけに、関数の異なるバージョンを宣言する必要がなくなるため、複数のオーバーロードが必要になるケースが減ります。

Set a default value by appending `=` to the parameter declaration:

```
fun read(  
    b: ByteArray,  
    // 「off」のデフォルト値は 0 です  
    off: Int = 0,  
    // 『len』のデフォルト値が計算されます  
    // 『b』配列のサイズとして  
    len: Int = b.size,  
) { /*...*/ }
```

デフォルト値が設定されているパラメータを、デフォルト値が設定されていないパラメータよりも先に宣言する場合、そのデフォルト値を使用するには、[引数に名前を付ける](#)必要があります：

```
fun greeting(  
    userId: Int = 0,  
    message: String,  
) { /*...*/ }
```

```
fun main() {  
    // Uses 0 as the default value for 'userId'  
    greeting(message = "Hello!")  
  
    // Error: No value passed for parameter 'message'  
    greeting("Hello!")  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" validate="false" id="default-before-ordinary"}
```

[末尾のラムダ](#)はこの規則の例外となります。これは、最後の引数が渡された関数に対応していなければならないためです：

```
fun main () {  
    //sampleStart  
    fun greeting(  
        userId: Int = 0,  
        message: () -> Unit,  
    )  
    { println(userId)  
      message() }  
  
    // 『userId』にデフォルト値を使用します  
    greeting() { println ("Hello!") }  
    // 0  
    // Hello!  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="default-before-trailing-lambda"}
```

メソッドのオーバーライド では、常に基底メソッドのデフォルトのパラメータ値が使用されます。デフォルトのパラメータ値を持つメソッドをオーバーライドする場合は、シグネチャからそのデフォルトのパラメータ値を省略する必要があります：

```
open class Shape {
    open fun draw(width: Int = 10, height: Int = 5) { /*...*/ }
}

class Rectangle : Shape() {
    // ここではデフォルト値を指定することはできません
    // ただし、この関数もデフォルトでは「width」に10、「height」に5を設定しています。
    override fun draw(width: Int, height: Int) { /*...*/ }
}
```

Non-constant expressions as default values

デフォルト値としての非定数式

パラメータには、定数ではないデフォルト値を割り当てることができます。たとえば、デフォルト値は、関数呼び出しの結果であったり、他の引数の値を用いた計算の結果であったりします。この例にある `len` パラメータがその一例です：

```
fun read(
    b: ByteArray,
    off: Int = 0,
    len: Int = b.size,
) { /*...*/ }
```

他のパラメータの値を参照するパラメータは、その順序の後ろで宣言する必要があります。この例では、`len` は `b` の後に宣言する必要があります。

一般的に、パラメータのデフォルト値には、どのような式でも指定することができます。ただし、デフォルト値は、関数が対応する引数を **指定せずに** 呼び出され、デフォルト値を割り当てる必要がある場合にのみ評価されます。たとえば、この関数は、`print` パラメータを指定せずに呼び出された場合にのみ、1行を出力します：

```
fun main() {
    //sampleStart
    fun read(
        b: Int,
        print: Unit? = println("『print』 に対して引数が指定されていません")
    ) { println(b) }

    // 「『print』 に引数が渡されていません」と表示し、続いて「1」を表示する
    read(1)
}
```

```

// 「1」のみを出力します
read(1, null)

// ABCと表示し、続いて「1」と表示する (訳者追記)
read(1,println("ABC")) // (訳者追記)

//sampleEnd
}

```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="non-constant-default"}
```

関数宣言の最後のパラメータが関数型である場合、対応する **ラムダ** 引数を、名前付き引数として括弧内()、あるいは **括弧()の外** から、または括弧を省略して、渡すことができます：

```

fun main() {
    //sampleStart
    fun log(
        level: Int = 0,
        code: Int = 1,
        action: () -> Unit,
    ) { println (level)
        println (code)
        action() }

    // 「level」には 1 を指定し、「code」にはデフォルト値の 1 を使用します
    log(1) { println("Connection established1") }

    // 「level」には0、「code」には1という、両方のデフォルト値を使用します
    log(action = { println("Connection established2") })

    // 前の呼び出しと同等であり、両方のデフォルト値を使用し、actionに与える無名関数を()
    // 括弧外に置く (訳者追記)
    log() { println("Connection established3") } // (訳者追記)

    // さらに括弧()を省略してactionに与える無名関数のみを与える
    log { println("Connection established4") }

    //sampleEnd
}

```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="lambda-outside-parentheses"}
```

Named arguments

名前付き引数

関数を呼び出す際、その引数のうち1つまたは複数に名前を付けることができます。関数呼び出しの引数が多数ある場合、これは役立つことがあります。このような場合、引数に値を関連付けるのは困難です。特に、その引数が **null** やブール値である場合はなおさらです。

関数呼び出しで名前付き引数を使用する場合、それらの順序は任意に指定できます。

デフォルト値を持つ 4 つの引数を持つ `reformat()` 関数について考えてみましょう。

```
fun reformat(  
    str: String,  
    normalizeCase: Boolean = true,  
    upperCaseFirstLetter: Boolean = true,  
    divideByCamelHumps: Boolean = false,  
    wordSeparator: Char = ' ',  
) { /*...*/ }
```

この関数を呼び出す際、引数のいくつかに名前を付けることができます：

```
reformat(  
    "String!",  
    normalizeCase = false,  
    upperCaseFirstLetter = false,  
    divideByCamelHumps = true,  
    ' _'  
)
```

デフォルト値が設定されている引数はすべて省略できます：

```
reformat("This is a long String!")
```

また、引数をすべて省略するのではなく、デフォルト値が設定されている一部の引数だけを省略することもできます。ただし、最初の引数を省略した後は、それ以降の引数にはすべて名前を付ける必要があります：

```
reformat(  
    "This is a short String!",  
    upperCaseFirstLetter = false,  
    wordSeparator = ' _'  
)
```

対応する引数に名前を付けることで、[可変長引数](#) (`vararg`) を渡すことができます。この例では、配列になっています：

```
fun mergeStrings(vararg strings: String) { /*...*/ }  
  
mergeStrings(strings = arrayOf("a", "b", "c"))
```

JVM上でJava関数を呼び出す場合、Javaバイトコードでは関数パラメータの名前が必ずしも保持されるわけではないため、名前付き引数の構文を使用することはできません。

{style="note"}

Return types

戻り値の型

ブロック本体（中括弧 `{}` で囲まれた文）を持つ関数を宣言する場合、必ず戻り値の型を明示的に指定する必要があります。 唯一の例外は、関数が `Unit` を返す場合であり、[その場合は戻り値の型を指定することは任意です](#)。

Kotlin では、ブロック本体を伴う関数の戻り値型は推論されません。 その制御フローは複雑になることがあり、その結果、読み手にとって、時にはコンパイラにとっても、戻り値の型が不明確になってしまうことがあります。 ただし、Kotlin では、[単一式関数](#) について、戻り値の型を明示的に指定しなくても、型推論によって推定することができます。

Kotlinの関数は単一の値を返しますが、その値には複数のデータが含まれることがあります。 これらの値を表現する方法については、「複数の値を返す」(#return-multiple-values) を参照してください。

Return multiple values

複数の値を返す

意味が異なる複数の関連する値を返す必要がある場合は、たとえ1つの関数でのみ使用するとしても、[データクラス](#)を宣言してください：

```
data class OrderSummary(  
    val subtotal: Double,  
    val tax: Double,  
)  
  
fun calculateOrderSummary(prices: List<Double>): OrderSummary {  
    val subtotal = prices.sum()  
    val tax = subtotal * 0.2  
    return OrderSummary(subtotal, tax)  
}  
  
fun main() {  
    val summary = calculateOrderSummary(listOf(12.50, 8.00, 4.50))  
  
    println(summary.subtotal)  
    // 25.0  
    println(summary.tax)  
    // 5.0  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="return-multiple-values-data-class"}

データクラスは、意味の異なる複数の値を返す必要がある場合に有効です。返される値が同じ種類であり、それらを一括して処理したい場合は、代わりにコレクションを返すことを検討してください：

```
data class Person(val name: String)

val friendGroups = listOf(
    listOf(Person("Alice"), Person("Bob")),
    listOf(Person("Charlie"), Person("Diana"), Person("Eve")),
    listOf(Person("Frank"))
)

fun findLargestGroupOfFriends(): List<Person> {
    return friendGroups.maxByOrNull { it.size } ?: emptyList()
}

fun main() {
    val largestGroup = findLargestGroupOfFriends()

    println(largestGroup.map { it.name })
    // [Charlie, Diana, Eve]
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.4" id="return-multiple-values-list"}

固定数の値を返す必要がある場合は、標準ライブラリの `Pair` や `Triple` といったデータクラスを使用することもできます。しかし、それらのプロパティには `first`、`second`、`third` といった一般的な名前が付けられているため、結果がわかりにくくなる可能性があります。

たとえば、`calculateOrderTotals()` 関数は `Pair` を返しますが、それぞれの `Double` が何を表しているのかが明確ではありません：

```
fun calculateOrderTotals(prices: List<Double>): Pair<Double, Double> {
    val subtotal = prices.sum()
    val tax = subtotal * 0.2
    return Pair(subtotal, tax)
}

fun main() {
    val totals = calculateOrderTotals(listOf(12.50, 8.00, 4.50))

    // What does 'first' mean?
    println(totals.first)
    // 25.0

    // What does 'second' mean?
    println(totals.second)
    // 5.0
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="return-multiple-values-pair"}
```

意味が明確に区別される結果を得るためには、[OrderSummary データクラスの例](#)で示されているように、説明的なプロパティ名を持つデータクラスを使用することをお勧めします。

Single-expression functions

単一式関数

関数の本体が単一の式で構成されている場合、中括弧を省略し、`=` 記号の後に本体を指定することができます：

```
fun twice(x: Int): Int = x * 2
```

ほとんどの場合、[戻り値の型](#)を明示的に宣言する必要はありません：

```
// コンパイラは、この関数が Int を返すものと推論する
fun double(x: Int) = x * 2
```

コンパイラは、単一の式から戻り値の型を推論する際に、問題が発生することがあります。そのような場合は、戻り値の型を明示的に指定する必要があります。たとえば、再帰的または相互再帰的（互いに呼び合う）関数や、`fun empty() = null` のような型のない式を含む関数では、常に戻り値の型を指定する必要があります。

推論された戻り値型を使用する場合は、コンパイラがユーザーにとってあまり有用ではない型を推論してしまう可能性があるため、必ず実際の結果を確認するようにしてください。上記の例で、`twice()` 関数が `Int` ではなく `Number` を返すようにしたい場合は、これを明示的に宣言する必要があります。

式本体内で `return` 文を使用する場合は、戻り値の型を明示的に指定する必要があります：（訳者注：このコードはKotlin Ver.2.4以上で作動します。）

```
fun getDisplayNameOrDefault(userId: String?): String =
    getDisplayName(userId ?: return "default")
```

Unit-returning functions

Unitを返す関数

関数にブロック本体（中括弧 `{}` で囲まれた命令）があり、有用な値を返さない場合、コンパイラはその戻り値の型が `Unit` であるとみなします。 `Unit` は、値が 1 つしかない型であり、これも `Unit` と呼ばれます。

関数型のパラメータを除き、戻り値の型として `Unit` を指定する必要はありません。 `Unit` を明示的に返さなくても構いません。

たとえば、`Unit`を返さない[printHello\(\)](#)関数を宣言することができます：

```
// 関数型パラメータ（『action』）の宣言には、依然として明示的な戻り値型が必要です
fun printHello(name: String?, action: () -> Unit) {
    if (name != null)
        println("こんにちは $name")
    else
        println("Hi there!")

    action()
}

fun main() {
    printHello("Kodee") {
        println("これは挨拶の後に再生されます。")
    }
    // こんにちは Kodee
    // これは挨拶の後に表示されます。

    printHello(null) {
        println("名前が指定されていなくても、アクションは実行されます。")
    }
    // Hi there(こんにちは)!
    // 名前は指定されていないが、アクションは正常に実行されます。
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" validate="false" id="return-unit-implicit"}

これは、次のような冗長な宣言と同等です：

```
//sampleStart
fun printHello(name: String?, action: () -> Unit): Unit {
    if (name != null)
        println("こんにちは $name")
    else
        println("Hi there(こんにちは)!")

    action()
    return Unit    // 訳者注) 冗長な部分
}
//sampleEnd
fun main() {
    printHello("Kodee") {
        println("これは挨拶の後に再生されます。")
    }
    // こんにちは Kodee
    // これは挨拶の後に表示されます。

    printHello(null) {
        println("名前が指定されていなくても、アクションは実行されます。")
    }
}
```

```
// 名前は指定されていないが、アクションは正常に実行されます。  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" validate="false" id="return-unit-explicit"}
```

可変長引数 (varargs)

関数に可変長の引数を渡すには、そのパラメータのうちの1つ（通常は最後のもの）に **vararg** 修飾子を指定します。関数内では、型 **T** の **vararg** パラメータを **T** の配列として使用できます：

```
fun <T> asList(vararg ts: T): List<T> {  
    val result = ArrayList<T>()  
    for (t in ts) // これは配列です  
        result.add(t)  
    return result  
}
```

そうすれば、関数に可変個の引数を渡すことができます：

```
fun <T> asList(vararg ts: T): List<T> {  
    val result = ArrayList<T>()  
    for (t in ts) // ts is an Array  
        result.add(t)  
    return result  
}  
  
fun main() {  
    //sampleStart  
    val list = asList(1, 2, 3)  
    println(list)  
    // [1, 2, 3]  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" validate="false" id="varargs-aslist"}
```

vararg として指定できるパラメータは1つだけです。パラメータリストの最後以外で **vararg** パラメータを宣言する場合は、その後に続くパラメータの値を名前付き引数を使用して渡す必要があります。

(以下のコードは訳者追記)

```
fun <T> asList(vararg ts: T, param1: Int, param2: String): List<T> {  
    val result = ArrayList<T>()  
  
    for (t in ts) // ts is an Array  
        result.add(t)
```

```

        result.add(param1 as T)
        result.add(param2.length as T)

    return result
}

fun main() {
    //sampleStart
    val list = asList(1, 2, 3, param1 = 100, param2 = "bbbb")

    println(list)
    // [1, 2, 3, 100, 4]
    //sampleEnd
}

```

(訳者追記ここまで)

パラメータの型が関数型である場合、括弧の外側にラムダ式を記述することで、その値を渡すこともできます。（次のコードについて訳者追記）(GoogleAiにより作成)

```

// 1. 関数の定義
// vararg を先に書き、最後の引数に関数型 (block) を配置する
fun <T> processItems(vararg items: T, block: (T) -> Unit) {
    for (item in items) {
        block(item)
    }
}

fun main() {
    // 2. 関数の呼び出し
    // 可変長引数を括弧の中に書き、関数型の引数（ラムダ式）を括弧の外に出す
    processItems("Apple", "Banana", "Cherry") { item ->
        println("フルーツ: $item")
    }
}

```

(訳者追記ここまで)

`vararg` 関数を呼び出す際は、`asList(1, 2, 3)` の例のように、引数を個別に渡すことができます。すでに配列があり、その内容を `vararg` パラメータとして、あるいは関数の一部として関数に渡したい場合は、配列名の前に `*` を付けて [スプレッド演算子](#) を使用します：

```

fun <T> asList(vararg ts: T): List<T> {
    val result = ArrayList<T>()
    for (t in ts)
        result.add(t)
    return result
}

```

```
fun main() {
    //sampleStart
    val a = arrayOf(1, 2, 3)

    // この関数は、配列 [-1, 0, 1, 2, 3, 4] を受け取ります
    list = asList(-1, 0, *a, 4)

    println(list)
    // [-1, 0, 1, 2, 3, 4]
    //sampleEnd
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" validate="false" id="varargs-aslist-with-array"}
```

プリミティブ型の配列を `vararg` として渡したい場合は、`.toTypedArray()` 関数を使用して、それを通常の（型付き）配列に変換する必要があります：

```
// 「a」は IntArray であり、これはプリミティブ型の配列です。
val a = intArrayOf(1, 2, 3)
val list = asList(-1, 0, *a.toTypedArray(), 4)
```

Infix notation

インフィックス表記

`infix` キーワードを使用すると、括弧やピリオドを付けずに呼び出せる関数を宣言することができます。これにより、コード内の単純な関数呼び出しの可読性を高めることができます。

```
infix fun Int.shl(x: Int): Int { /*...*/ }

// 一般的な表記法を使用して関数を呼び出します
1.shl(2)

// インフィックス表記を使用して関数を呼び出す
1 shl 2
```

インフィックス関数は、以下の要件を満たさなければなりません：

- それらは、クラスのメンバ関数、または [拡張関数](#) でなければなりません。
- パラメータは1つでなければならない。
- パラメータは、[可変長の引数](#) (`vararg`) を受け入れてはならず、また [デフォルト値](#) を持つてはなりません。

インフィックス関数呼び出しは、算術演算子、型キャスト、および `rangeTo` 演算子よりも優先順位が低くなります。以下の式は等価です：

- `1 shl 2 + 3` は `1 shl (2 + 3)` と同等です

- `0 until n * 2` は `0 until (n * 2)` と同等です。
- `xs union ys as Set<*>` は `xs union (ys as Set<*>)` と同等です。

一方、中置関数呼び出しの優先順位は、ブール演算子 `&&` や `||`、`is` や `in` によるチェック、およびその他のいくつかの演算子よりも高い。以下の式も同様に等価です：

- `a && b xor c` は `a && (b xor c)` と同等です。
- `a xor b in c` は `(a xor b) in c` と同等です。

{style="note"}

インフィックス関数では、必ずレシーバーとパラメータの両方を指定する必要があることに注意してください。インフィックス表記を使用して現在のレシーバーに対してメソッドを呼び出す場合は、明示的に `this` を使用してください。これにより、曖昧さのない構文解析が保証されます。

```
class MyStringCollection {
    val items = mutableListOf<String>()

    infix fun add(s: String) {
        println("Adding: $s")
        items += s
    }

    fun build() {
        add("first")           // 正しい：通常の関数呼び出し
        this add "second"     // 正しい：明示的なレシーバーを伴うインフィックス呼び出し
        // add "third"        // Compiler error: 明示的なレシーバーが必要です（addの前に
thisが不足している）
    }

    fun printAll() = println("Items = $items")
}

fun main() {
    val myStrings = MyStringCollection()
    // Adds "first" and "second" to the list
    myStrings.build()

    myStrings.printAll()
    // Adding: first
    // Adding: second
    // Items = [first, second]
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="infix-notation-example"}

Function scope

関数のスコープ

Kotlinの関数は、ファイルの最上位で宣言することができます。つまり、関数を格納するためのクラスを作成する必要はありません。関数は、`_メンバ関数_`や`_拡張関数_`としてローカルに宣言することもできます。

Local functions

ローカル関数

Kotlin は、他の関数内で宣言される関数であるローカル関数をサポートしています。たとえば、次のコードは、与えられたグラフに対して深さ優先探索アルゴリズムを実装したものです。実装を隠蔽し、再帰呼び出しを処理するために、外側の `dfs()` 関数内にローカルな `dfs()` 関数を定義します：

```
class Person(val name: String) {
    val friends = mutableListOf<Person>()
}
class SocialGraph(val people: List<Person>)
//sampleStart
fun dfs(graph: SocialGraph) {
    fun dfs(current: Person, visited: MutableSet<Person>) {
        if (!visited.add(current)) return
        println("Visited ${current.name}")
        for (friend in current.friends)
            dfs(friend, visited)
    }
    dfs(graph.people[0], HashSet())
}
//sampleEnd
fun main() {
    val alice = Person("Alice")
    val bob = Person("Bob")
    val charlie = Person("Charlie")
    alice.friends += bob
    bob.friends += charlie
    charlie.friends += alice
    val network = SocialGraph(listOf(alice, bob, charlie))
    dfs(network)
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="local-functions-dfs"}

実装を隠蔽し、再帰呼び出しを処理するために、外側の `dfs()` 関数内にローカルな `dfs()` 関数を定義します：上記の例では、`visited` 関数の引数はローカル変数にすることができます：

```
class Person(val name: String) {
    val friends = mutableListOf<Person>()
}
class SocialGraph(val people: List<Person>)
//sampleStart
fun dfs(graph: SocialGraph) {
    val visited = HashSet<Person>()
    fun dfs(current: Person) {
```

```

        if (!visited.add(current)) return
        println("Visited ${current.name}")
        for (friend in current.friends)
            dfs(friend)
    }
    dfs(graph.people[0])
}
//sampleEnd
fun main() {
    val alice = Person("Alice")
    val bob = Person("Bob")
    val charlie = Person("Charlie")
    alice.friends += bob
    bob.friends += charlie
    charlie.friends += alice
    val network = SocialGraph(listOf(alice, bob, charlie))
    dfs(network)
}

```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="local-functions-dfs-with-local-variable"}
```

Member functions

メンバー関数

メンバー関数とは、クラスやオブジェクトの内部で定義される関数のことです：

```

class Sample {
    fun foo() { print("Foo") }
}

```

メンバ関数を呼び出すには、インスタンス名またはオブジェクト名を記述し、その後に `.` を付けて関数名を記述します。

```

// Streamクラスのインスタンスを作成し、read()を呼び出します
Stream().read()

```

クラスおよびメンバのオーバーライドに関する詳細については、[クラス](#)および[継承](#)を参照してください。

Generic functions

汎用関数

関数の前に山括弧 `<>` を付けることで、その関数のジェネリックパラメータを指定できます：

```

fun <T> singletonList(item: T): List<T> { /*...*/ }

```

ジェネリック関数に関する詳細については、[ジェネリック](#)を参照してください。

Tail recursive functions

末尾再帰関数

Kotlin は、[末尾再帰](#)として知られる関数型プログラミングのスタイルをサポートしています。通常はループを使用するアルゴリズムの中には、スタックオーバーフローのリスクを伴わずに、代わりに再帰関数を使用できるものがあります。関数が `tailrec` 修飾子でマークされ、必要な形式条件を満たしている場合、コンパイラは再帰を最適化して削除し、代わりに高速で効率的なループベースのバージョンを生成します：

```
import kotlin.math.cos
import kotlin.math.abs

// 「十分に小さい」と言える任意の精度
val eps = 1E-10

tailrec fun findFixPoint(x: Double = 1.0): Double =
    if (abs(x - cos(x)) < eps) x else findFixPoint(cos(x))
```

このコードは、コサインの固定点（数学上の定数）を計算します。この関数は、結果が変化しなくなるまで `1.0` から `cos()` を繰り返し呼び出し、指定された `eps` の精度で `0.7390851332151611` という結果を返します。このコードは、次のようなより伝統的なスタイルと同等です：

```
import kotlin.math.cos
import kotlin.math.abs

// 「十分に小さい」と言える任意の精度
val eps = 1E-10

private fun findFixPoint(): Double {
    var x = 1.0
    while (true) {
        val y = cos(x)
        if (abs(x - y) < eps) return x
        x = cos(x)
    }
}
```

関数に `tailrec` 修飾子を適用できるのは、その関数が最後の操作として自分自身を呼び出す場合に限りです。再帰呼び出しの後にコードが続く場合、`try/catch/finally` ブロック内にある場合、または関数が `オープン` である場合は、末尾再帰を使用することはできません。

See also:

- [Inline functions](#)
- [Extension functions](#)
- [Higher-order functions and lambdas](#)